

LAB 2

Bu haftaki lab etkinliğimizde projelerimiz için genel araştırma yaptık ve bir akış şeması oluşturmaya çalıştık. Akış şemasını aşağıdaki başlıklar çerçevesinde oluşturmaya çalıştım.

1. Simülasyon aracı
2. Donanım kurulumu
3. Veri toplama
4. ML modeli
5. Sistem entegrasyonu
6. Performans değerlendirmesi

PROJENİN AMACI

LoRaWAN ağında çalışan bir düğümün (node) enerji tüketimi, veri gönderim sıklığı veya sinyal kalitesi (RSSI/SNR) gibi parametrelerini yapay zekâ ile optimize etmek.

Proje Fikirleri (AI destekli LoRa/WiFi optimizasyonu)

- Enerji optimizasyonu
- Kanal seçimi
- WiFi–LoRa akıllı geçiş

1. Enerji Tüketimi Optimizasyonu (Battery-Aware AI Node)

Amaç:

Node'un pil ömrünü artırmak için veri gönderim sıklığını ve iletim gücünü (tx_power) ML modeliyle dinamik olarak ayarlamak.

Örnek:

- Node, ortalama RSSI ve pil seviyesine göre en uygun veri gönderme aralığını tahmin eder.
- Zayıf sinyal varsa: veri sıklığını azaltır.
- Güçlü sinyal varsa: daha sık gönderim yapar.

Girdi verileri: RSSI, SNR, pil seviyesi

Çıktı: Gönderim aralığı (interval) veya TX gücü

ML modeli: Linear Regression, Random Forest veya TinyML tabanlı sinir ağı

2. Kanal ve Frekans Optimizasyonu (AI-based Channel Selector)

Amaç:

LoRa kanal gürültüsünü (noise/interference) analiz ederek en az yoğun kanalı seçen bir yapay zekâ modeli oluşturmak.

Örnek:

- LoPy4, birkaç kanaldan test sinyalleri alır.
- ML modeli, hangi kanalda SNR en yüksekse onu seçer.
- Gürültü koşullarına göre kanal seçimi dinamik olarak değişir.

Girdi verileri: Kanal frekansı, RSSI, SNR, paket kaybı oranı

Çıktı: En uygun kanal frekansı

ML modeli: Decision Tree, KNN

3. LoRa vs WiFi Akıllı Seçim (Hybrid Node Optimization)

Amaç:

AI modeli, veri büyüklüğüne ve bağlantı kalitesine göre WiFi mi LoRa mı kullanılacağına karar verir.

Örnek:

- Küçük veriler: LoRa (enerji verimli)
- Büyük veriler ve yakın mesafe: WiFi (hızlı)
- Model, ortam koşullarına (sinyal gücü, gecikme, enerji tüketimi) göre karar verir.

Girdi verileri: RSSI (LoRa/WiFi), veri boyutu, pil seviyesi

Çıktı: “WiFi” veya “LoRa” seçimi

ML modeli: Logistic Regression veya basit Neural Network

1. Simülasyon Aracı

LoRaWAN Ağ Simülatörleri

Bu yöntemler doğrudan LoPy4 yerine LoRa ağını simüle eder:

Simülatör	Açıklama	Kullanım Amacı
LoRaSim (Python)	LoRaWAN MAC ve PHY katmanını taklit eder.	Paket kaybı, enerji, SF optimizasyonu.
FLoRa (OMNeT++ tabanlı)	LoRaWAN ağını detaylı simüle eder.	RL veya optimizasyon testi.
NS-3 LoRaWAN module	Gerçekçi ağ topolojisi testleri.	Akademik düzey ağ optimizasyonu.

2. Donanım Bağlantısı

1. LoPy4'ü Expansion Board'a tak

- Kartın pinleri expansion board üzerindeki yuvalara denk gelmeli.
- Bastırırken dikkat et, pinler yamulmasın.
- “LoRa” yazan yerin yanındaki anten konnektörüne anteni tak.

Uyarı:

LoRa antenini takmadan LoRa modülünü çalıştırma!

RF enerjisi boşta kalırsa donanım kalıcı olarak zarar görebilir.

2. USB ile bağla

- Expansion Board üzerindeki Micro USB portunu bilgisayara bağla.
- Güç LED'i (yeşil) yanıyorsa bağlantı tamamdır.
- Cihaz seri port (COM port) olarak görünmelidir.

3. Yazılım Kurulumu (Bilgisayar Tarafı)

1. Pycom Firmware Update Tool

Bu araç, LoPy4'e son sürüm MicroPython firmware'ini yükler.
<https://software.pycom.io/>

- Uygulamayı indir → “LoPy4” modelini seç
- USB portunu seç
- “Erase and update firmware” seçeneğiyle güncelle

2. REPL (Python konsolu) bağlantısı

LoPy4 üzerinde doğrudan Python kodu çalıştırabilirsin.
Bağlanmak için:

Windows:

- PuTTY veya Tera Term kullan
- Port: COMx
- Baudrate: 115200

VS Code + Pymakr Eklentisi

- VS Code'a “Pymakr” eklentisini kur
- Cihazı otomatik algılar
- Python kodunu doğrudan kartta çalıştırabilir, yükleyebilir, REPL açabilirsin

4. LoRa Testi (Temel Kod)

LoRa antenini taktıktan sonra şu basit kodla LoRa modülünü test edebilirsin

Gönderici Node:

```
from network import LoRa

import socket

import time

import binascii

# LoRa modunu başlat

lora = LoRa(mode=LoRa.LORA, region=LoRa.EU868)

s = socket.socket(socket.AF_LORA, socket.SOCK_RAW)
```

```
while True:

    s.setblocking(True)

    s.send(b'hello from LoPy4')

    print("Mesaj gönderildi")

    time.sleep(5)
```

Alıcı Node:

```
from network import LoRa

import socket

lora = LoRa(mode=LoRa.LORA, region=LoRa.EU868)

s = socket.socket(socket.AF_LORA, socket.SOCK_RAW)

while True:

    s.setblocking(False)

    data = s.recv(64)

    if data:

        print("Alındı:", data)
```

Bu iki kodu iki farklı LoPy4 üzerinde çalıştırırsan, aralarındaki LoRa iletişimini görebilirsin.

5. WiFi Testi

LoPy4 aynı zamanda WiFi'yi de destekler:

```
from network import WLAN

wlan = WLAN(mode=WLAN.STA)

wlan.connect(ssid="wifi_adi", auth=(WLAN.WPA2, "sifre"))

while not wlan.isconnected():

    pass

print("WiFi bağlı:", wlan.ifconfig())
```

Bu kod LoPy4'ü bir WiFi ağına bağlar.

3. Veri Toplama

Toplayabileceğin veriler; donanımsal sensörlerden gelen fiziksel ölçümler (örneğin sıcaklık, pil durumu) ve haberleşme katmanından gelen ağ metrikleri (örneğin RSSI, SNR, paket gecikmesi) olarak ikiye ayrılır.

LoPy4'ten Alabilecek Veriler

Kategori	Örnek Veriler
LoRa Katmanı	RSSI, SNR, SF, CR, TX Power, Frequency, Payload Size, Packet Loss, CRC Errors, AirTime
Enerji	Pil seviyesi, akım tüketimi, çalışma süresi, deep sleep sayısı
WiFi Katmanı	RSSI, bağlantı kalitesi, throughput, latency
Sistem	RAM, CPU frekansı, işlem yükü
Sensör/Ortam	Sıcaklık, nem, basınç, ışık, konum
Zamanlama	RTT, paket aralığı, işlem gecikmesi

4. ML Modeli

Bu aşamada Pycom LoPy4 gibi gömülü bir cihazda yapay zeka (ML) modeli çalıştırmak için, hem donanım kısıtlarını (RAM, işlemci, enerji) hem de veri türünü (örneğin LoRa RSSI, SNR, paket kaybı vs.) dikkate almak gerekiyor.

Genel Durum:

LoPy4 üzerinde tam bir ML modeli eğitmek mümkün değil (çünkü cihazın hafızası ve işlem gücü sınırlı).

Ama önceden eğitilmiş küçük bir modeli çalıştırmak veya basit karar modelleri oluşturmak mümkün.

Yani 2 yol var:

1. Model eğitimi bilgisayarda / bulutta
2. Modelin basitleştirilmiş sürümünü LoPy4'e yükleme (inference)

LoPy4'te veya çevresinde kullanılabilecek ML türleri:

Amaç	Uygun Model Türü	Açıklama
Enerji tüketimi optimizasyonu	Decision Tree, Linear Regression	Pil seviyesine ve sinyal gücüne göre gönderim sıklığını ayarlayabilir.
Bağlantı kalitesi tahmini (RSSI/SNR)	Random Forest, k-NN, TinyML modeli	Gelecekteki sinyal kalitesini tahmin edebilir.
Veri iletim hızı optimizasyonu	Reinforcement Learning (Q-learning)	Kanal koşullarına göre adaptif hız seçimi yapılabilir.
Paket kaybı tahmini	Logistic Regression, Naive Bayes	Paket kaybı oranını tahmin edip yeniden gönderim politikası belirlenebilir.
Edge AI (TinyML)	TensorFlow Lite Micro model	Çok basit sinir ağı (örneğin 1 gizli katmanlı) LoPy4'te doğrudan çalıştırılabilir.

Uygulanabilir ML kütüphaneleri:

1. TinyML / TensorFlow Lite Micro

- Modeli PC'de TensorFlow ile eğitip .tflite formatına dönüştürürsün.
- Ardından micro versiyonu LoPy4 üzerinde C veya MicroPython ile çalıştırılır.
- Kullanım: sinyal kalitesi sınıflandırma, veri oranı tahmini.

2. Edge Impulse

- Tarayıcı tabanlı bir platform, LoPy4 ile entegre edilebilir.
- Sensör verilerini toplar, bulutta model eğitir, optimize edilmiş model dosyası üretir.

3. Basit Python tabanlı modeller (LoPy'de)

- LoPy4 üzerinde MicroPython kullanıldığı için basit modelleri (örneğin lineer regresyon formülünü) direkt olarak elle uygulayabilirsin.
- Örnek: RSSI ve SNR'ye göre "paket kaybı olasılığı" tahmini yapan bir denklem.

5. Sistem Entegrasyonu

Bu aşamada amaç:

Toplanan verinin modele entegre edilmesi, model çıktısının LoRaWAN veya WiFi üzerinden kullanılabilir hale getirilmesi, sistemin uçtan uca çalışır hale gelmesi.

Entegrasyonun Temel Katmanları:

Katman	Açıklama	Kullanılacak Teknolojiler
1. Donanım Katmanı	LoPy4 + sensörler (ör. pil sensörü, RSSI ölçümü)	Pycom LoPy4, MicroPython
2. İletişim Katmanı	LoRaWAN veya WiFi üzerinden veri gönderimi	LoRa (The Things Network) veya WiFi MQTT
3. Bulut/Server Katmanı	Verilerin toplandığı, işlendiği ve modelin eğitildiği yer	Python, scikit-learn, TensorFlow, Edge Impulse
4. Model Entegrasyonu Katmanı	Eğitilmiş modelin LoPy4'e gömülmesi	TensorFlow Lite Micro veya manuel formül
5. Karar / Optimizasyon Katmanı	Model çıktısına göre dinamik parametre değişimi	MicroPython kontrol kodu

Sistem Diyagramı

[Sensörler/LoPy4] → [Veri Toplama] → [LoRaWAN/WiFi] → [Sunucu]

↑

↓

[Basit ML Model / Decision Logic] ← [Model Eğitimi ve Güncelleme]

6. Performans Değerlendirmesi

AI destekli LoRaWAN node sisteminin, klasik (AI olmayan) sistemlere göre:

- daha verimli enerji kullanıp kullanmadığını,
- paket kaybını azaltıp azaltmadığını,
- iletişim kalitesini artırıp artırmadığını ölçmek ve kanıtlamak

Değerlendirme Metrikleri

Metrik	Açıklama	Ölçüm Yöntemi
Paket Teslim Oranı (Packet Delivery Ratio - PDR)	Gönderilen paketlerin başarılı bir şekilde ulaşıp ulaşmadığı	$\frac{\text{alınan_paket}}{\text{gönderilen_paket}} * 100$
Ortalama RSSI ve SNR	İletişim kalitesini gösterir	LoRa istatistiklerinden (lora.stats()) alınır
Paket Gecikmesi (Latency)	Gönderimden alıma kadar geçen süre	Zaman damgaları karşılaştırılır
Enerji Tüketimi (mAh veya % düşüş)	Pil seviyesi üzerinden izlenir	Pil sensörü veya gerilim ölçümü
Veri İletim Hızı (bps veya byte/s)	Kanal verimliliği	Gönderilen veri miktarı / süre
Model Karar Doğruluğu	ML modelinin doğru tahmin oranı	Gerçek ölçümle model kararını karşılaştırarak hesaplanır